

CPP09

C++ Templates, STL Containers & Algorithms

ex00 · BitcoinExchange | ex01 · RPN | ex02 · PmergeMe

ex00 — BitcoinExchange

What it does

Reads a CSV database of historical Bitcoin exchange rates and an input file of (date, amount) pairs. For each input line it finds the closest earlier rate in the database and prints `date => amount = result`.

Key concepts

`std::map`

An ordered associative container. Keys are kept sorted at all times. Internally a red-black tree. All operations (insert, find, lower_bound) are $O(\log n)$.

```
std::map<std::string, double> rates;
// keys are date strings "YYYY-MM-DD" - lexicographic order == chronological order
```

lower_bound — the core trick

`map::lower_bound(key)` returns an iterator to the first entry whose key is $\geq$ *key*. This lets us find the closest date without scanning:

```
auto it = rates.lower_bound(date);
// it points to the first date >= the requested date
if (it == rates.end() || it->first != date) {
    if (it == rates.begin()) { /* no earlier date exists */ }
    --it; // step back to the largest date that is still <= requested
}
// it->second is the correct exchange rate
```

Why strings work here: "YYYY-MM-DD" format sorts lexicographically the same as chronologically, so `lower_bound` on a string key gives the right date.

File parsing

- Database (`data.csv`): header `date,exchange_rate`, then comma-separated rows.
- Input: header `date | value`, then rows separated by `" | "`.
- Uses `std::getline` + `std::stringstream` for splitting.
- Errors on individual lines are printed to `stderr` and skipped; the program continues.
- Fatal errors (bad file, bad header) throw `std::runtime_error`.

Date validation

```
bool BitcoinExchange::isValidDate(const std::string& date) {
    // length must be 10, positions 4 and 7 must be '-'
    // remaining positions must be digits
    // month 1-12, day within days-in-month (leap year aware)
}
```

Value validation

- Must parse as a `double` with no trailing characters.
- Must be ≥ 0 and ≤ 1000 .
- NaN is rejected explicitly (`std::isnan`).

Class design

```
class BitcoinExchange {
public:
    BitcoinExchange(const std::string& path); // loads database
    void processInput(const std::string& filename);
```

```
private:
    std::map<std::string, double> rates;
    static bool isValidDate(const std::string& date);
    void loadDatabase(const std::string& filename);
};
```

Data flow

```
main(av[1]) → BitcoinExchange("data.csv") → loadDatabase → rates map built
              → processInput(av[1]) → for each line → lower_bound → print result
```

Common pitfalls

- Database errors are fatal (throw); input line errors are soft (cerr + continue).
- The header line of the input file must be consumed with `getline` before the loop.
- `--it` must only happen after checking `it != rates.begin()`.

ex01 — RPN (Reverse Polish Notation)

What it does

Evaluates a mathematical expression given in Reverse Polish Notation (postfix) passed as a single command-line argument. Prints the result or "Error".

What is RPN?

Operators come *after* their operands. No parentheses needed because the order is unambiguous.

Infix	RPN
3 + 4	3 4 +
(3 + 4) * 2	3 4 + 2 *
5 * (2 + 3) - 1	5 2 3 + * 1 -

Algorithm — stack machine

```
for each token in expression:
    if token is a number:
        push onto stack
    if token is an operator (+, -, *, /):
        pop two values (val1 = top, val2 = second)
        push (val2 OP val1)
at end: stack must contain exactly one value → that is the result
```

Order matters: `val1 = stack.top(); pop; val2 = stack.top(); pop;`
Then result = `val2 OP val1` . For subtraction and division this is critical.

std::stack

A LIFO (last-in, first-out) adapter. Wraps `std::deque` by default. Key operations: `push(x)` , `top()` , `pop()` , `size()` .

```
std::stack<long long> _stack;
_stack.push(42);
long long v = _stack.top(); // peek
_stack.pop();              // remove top (returns void)
```

Parsing

```
std::istringstream iss(input);
std::string token;
while (iss >> token) { /* process token */ }
```

`std::istringstream` lets you treat a string like a file stream, splitting on whitespace.

Error conditions

- Fewer than 2 values on stack when operator encountered → Error
- Division by zero → Error
- Stack does not have exactly 1 value at end → Error
- Token is not a valid number or operator → Error

Class design

```
class RPN {
    std::string _input;
    std::stack<long long> _stack;
public:
```

```
RPN(const std::string& input); // parses and evaluates in constructor  
};
```

The constructor does all the work — it evaluates the expression and prints the result. If anything goes wrong it throws `std::runtime_error("Error")` .

Data flow

```
main(av[1]) → RPN(av[1]) → tokenise → for each token → stack ops → cout result
```

ex02 — PmergeMe (Ford–Johnson Sort)

What it does

Sorts a sequence of positive integers using the Ford–Johnson (Merge-Insertion) algorithm — twice, once with `std::vector` and once with `std::deque` — and reports timing and comparison counts for each.

Ford–Johnson algorithm overview

Ford–Johnson achieves the theoretical information-theoretic lower bound $\lceil \log_2(n!) \rceil$ comparisons for many values of n , making it one of the most comparison-efficient sorting algorithms known.

The number of comparisons needed to sort n elements cannot be less than $\lceil \log_2(n!) \rceil$ because there are $n!$ possible orderings and each comparison rules out at most half of them.

n	Lower bound $\lceil \log_2(n!) \rceil$	Our result
5	7	7 ✓
11	26	26 ✓
21	66	66 ✓
22	70	70 ✓

Three phases (repeated at every recursion level)

Phase 2 — Descent

```
Input: [11, 2, 17, 0, 16, 8, ...]

Pair adjacent elements, compare each pair:
  (11,2)→ a=11, b=2    (17,0)→ a=17, b=0    (16,8)→ a=16, b=8 ...

Odd element (if sequence length is odd): set aside with no partner.

Recurse on a-sequence only: [11, 17, 16, ...]
```

Phase 3 — Unwind

```
Recursion returns sorted a-sequence: [a0, a1, a2, ...]

Build mainChain: [b1 | a0 a1 a2 ...]
                  ↑ free — b1 ≤ a0 guaranteed by pairing

Build pend: [{b2, bound=a1}, {b3, bound=a2}, ...]
Each b carries its a-partner as a future search boundary.
```

Phase 4 — Insertion

```
Insert pend elements using Jacobsthal order.
For each element:
  1. Find a-partner's current position in mainChain (binary search, uncounted)
  2. Binary search in [0, partner_position) — EVERY comparison counted
  3. Insert at found position
```

Jacobsthal sequence

Recurrence: $J(k) = J(k-1) + 2 \cdot J(k-2)$, starting 1, 3, 5, 11, 21, 43, 85, ...

Used to order the insertions so every element in a group faces exactly the same number of binary search comparisons. Groups are inserted high-to-low within each Jacobsthal bracket.

```

Group 1: b3, b2          (each costs 2 comparisons)
Group 2: b5, b4          (each costs 3 comparisons)
Group 3: b11,b10,...,b6  (each costs 4 comparisons)

```

Why Jacobsthal ordering saves comparisons

At the start of group k , the chain has exactly $2 \cdot J(k-1)$ elements before the boundary — a power of 2 minus 1. This means each element in the group fits into a binary search of depth exactly $\lceil \log_2(J(k)) \rceil$. Inserting from high to low within a group ensures no insertion shifts a later element's boundary rightward into the next power-of-2 bracket.

Data structures

Block and BlockPool

```

struct Block { int value; int rep() const { return value; } };
typedef int      BlockId;    // index into pool
typedef std::deque<Block>    BlockPool; // push_back-stable: no reallocation

```

`std::deque` is used for the pool because `push_back` never invalidates existing element addresses. `BlockIds` (integer indices) remain valid for the entire sort.

Pair preservation — the key invariant

Each recursion frame holds its own `pairs` vector on the C++ call stack. Deeper frames never touch shallower frames' pairs. At unwind time, each frame looks up its own pairs in $O(1)$ via a direct-address vector:

```

std::vector<BlockId> aToB(_pool.size(), INVALID_ID);
for (size_t i = 0; i < pairs.size(); i++)
    aToB[pairs[i].aId] = pairs[i].bId;

// lookup: aToB[someAId] → its b-partner, O(1), no scan

```

`BlockIds` are consecutive integers starting at 0, so the vector index is the key — a perfect hash. No `std::map` needed.

Sequence

```

typedef std::vector<BlockId> Sequence; // ordered list of block IDs

```

The algorithm works entirely on sequences of IDs. The actual values live in the pool and are never copied during sorting.

PairRecord and PendRecord

```

struct PairRecord { BlockId aId, bId; }; // one (a,b) bond at one depth
struct PendRecord { BlockId bId, boundId; }; // bId to insert, bounded by boundId

```

Template design

```

template <typename OutContainer>
class FordJohnson {
    // entire algorithm; OutContainer is std::vector<int> or std::deque<int>
public:
    OutContainer run(const std::vector<int>& input);
    int comparisons() const;
};

```

The algorithm is identical for both containers. The template parameter only affects the final output step where sorted values are written into the result.

Since `FordJohnson` is only instantiated inside `PmergeMe.cpp`, the template method definitions can live in the `.cpp` file and the compiler handles instantiation implicitly within the same translation unit.

Binary search — counted vs uncounted

Function	Purpose	Counted?
<code>findBoundPos</code>	Locate a-partner's current position in chain	No — bookkeeping
<code>binaryInsertPos</code>	Find insertion position in [o, bound)	Yes — every midpoint test

`findBoundPos` is not counted because it searches for an element already known to be sorted — it is repositioning a known value, not comparing an unknown one.

PmergeMe class

```
class PmergeMe {
    std::vector<int> _vec;
    std::deque<int> _deq;
public:
    void addValue(int val);
    void printBefore() const;
    void sortAndPrint();    // runs FordJohnson twice, prints timing
};
```

Stores values in both containers so both algorithm runs receive the same data.

Input validation (main.cpp)

- Must have at least one argument.
- Each argument parsed with `std::stoll`; must consume the entire string.
- Must be in range [0, INT_MAX].
- Duplicates detected with `std::set<int>` — rejected immediately.

Performance: vector vs deque

	std::vector	std::deque
Memory layout	Contiguous	Segmented blocks
Cache behaviour	Excellent	One indirection per block
Insert in middle	O(n) shift	O(n/2) shift
Comparison count	Identical	Identical
Speed (small n)	~equal	~equal
Speed (large n)	Faster	Slower

Vector wins at scale because cache locality outweighs deque's better asymptotic insert cost. The crossover typically appears around n=1000.

File structure

File	Contents
PmergeMe.hpp	FordJohnson template declaration (nested types + method signatures), PmergeMe class declaration
PmergeMe.cpp	All method implementations — FordJohnson template methods + PmergeMe methods
main.cpp	Input validation, PmergeMe usage
ALGORITHM.md	Extended algorithm documentation

Shared C++ Concepts Across All Exercises

Orthodox Canonical Form

All classes implement the four required members:

```
ClassName(); // default constructor
ClassName(const ClassName& o); // copy constructor
ClassName& operator=(const ClassName& o); // copy assignment
~ClassName(); // destructor
```

Required by the 42 norm. Ensures safe copying and assignment.

Exception handling

```
// throw
throw std::runtime_error("Error: message");

// catch
try { /* risky code */ }
catch (const std::exception& e) { std::cerr << e.what() << "\n"; }
```

Fatal errors propagate via exceptions. Per-line errors in ex00 and ex01 go to `stderr` and allow the program to continue.

String parsing idioms

```
// Split a string on whitespace
std::istringstream iss(str);
std::string token;
while (iss >> token) { /* ... */ }

// Split on a delimiter
std::string part;
std::getline(stream, part, ',');

// Parse a number, checking no trailing characters
size_t pos;
long long n = std::stoll(str, &pos);
if (pos != str.size()) { /* bad input */ }
```

Container choice summary

Container	Used in	Why
<code>std::map</code>	ex00	Sorted by key, $O(\log n)$ lower_bound
<code>std::stack</code>	ex01	LIFO for RPN evaluation
<code>std::vector</code>	ex02	Sequence storage, sorted output
<code>std::deque</code>	ex02	Stable pool + output container comparison
<code>std::set</code>	ex02 main	Duplicate detection in $O(\log n)$

Header vs implementation split

```
// .hpp – declarations only
class Foo {
    int _x;
public:
    Foo();
    void doSomething();
```

```
};

// .cpp – definitions
#include "Foo.hpp"
Foo::Foo() : _x(0) {}
void Foo::doSomething() { /* ... */ }
```

For template classes (ex02), method bodies must be visible when the template is instantiated. If only one `.cpp` uses the template, the definitions can live there alongside the usage — the compiler instantiates implicitly within the translation unit.

Makefile structure

```
CC      = c++
FLAGS   = -Wall -Werror -Wextra -std=c++11
SRC      = main.cpp Foo.cpp
OBJ      = $(SRC:.cpp=.o)
NAME     = program

all: $(NAME)

%.o: %.cpp
    $(CC) $(FLAGS) -c $< -o $@

$(NAME): $(OBJ)
    $(CC) $(FLAGS) $(OBJ) -o $(NAME)

clean:    $(RM) $(OBJ)
fclean:   clean && $(RM) $(NAME)
re:       fclean all
```